

Matthew Phillips

Full-Stack Product Engineer | Next.js / React / Node / FastAPI | Databases | AI Integrations | Production Systems

bajaracer415@gmail.com | 415-504-5897 | GitHub Portfolio: <https://github.com/Stericallyhindered/PORTFOLIO>

SUMMARY

Senior full-stack product engineer with deep experience building production web applications, Next.js/React frontends, Node/FastAPI backends, REST APIs, databases, admin systems, customer portals, automation tools, AI-integrated workflows, and hardware-connected software. I build systems that connect sales, support, operations, shipping, warranty, accounting-adjacent workflows, customer records, dashboards, and internal teams into usable software.

My strongest work is practical product engineering: taking messy real-world workflows, modeling the data, building the frontend and backend, integrating third-party APIs, shipping useful features quickly, and improving quality over time. I am fluent in JavaScript/TypeScript, React, Next.js, Node.js, Express, REST APIs, SQL/Postgres, SQLite, MongoDB/Mongoose, Payload CMS, Supabase, Python automation, CI/CD-style workflows, and advanced AI-assisted development with Claude, Codex, Cursor-style workflows, LM Studio, local LLMs, prompt engineering, code-architecture planning, codebase reverse engineering, context design, and custom automation pipelines.

ROLE FIT

- **Production web applications:** Built customer-facing portals, ecommerce/admin systems, CRM/order workflows, warranty/claims tools, dealer portals, dashboards, AI support systems, and internal operations tools.
- **JavaScript / frontend frameworks:** React, Next.js, TypeScript, JavaScript, component-driven UI, SPA-style dashboards, forms, tables, status screens, customer portals, admin panels, and responsive layouts.
- **Backend / APIs:** Node.js, Express, Next.js API routes, REST APIs, auth/session flows, JWT, role permissions, webhooks, file/document ingestion, notifications, payment/shipping integrations, and service-layer business logic.
- **Data modeling / SQL:** Postgres, SQLite, Supabase, Drizzle, Prisma-style workflows, MongoDB/Mongoose, Payload CMS, workflow state, audit trails, customer/order/device/support records, reporting, and operational dashboards.
- **Integration-heavy systems:** Stripe, UPS/shipping APIs, email/SMS providers, Resend/Nodemailer, Twilio, webhook flows, document/OCR-style intake, AI support connectors, and cross-system workflow automation.
- **Quality / ownership:** Bias toward action, readable code, clear data flow, edge-case handling, documentation, debugging, and practical testing/verification.
- **AI coding workflows:** Daily use of Claude, Codex, Cursor-style tools, LM Studio, local LLMs, custom agent workflows, prompt engineering, architecture planning, codebase reverse engineering, scripted context generation, guardrails, review loops, and automation pipelines for implementation, debugging, code review, documentation, refactoring, test/checklist generation, and codebase analysis while retaining responsibility for correctness.

CORE SKILLS

Frontend

- React, Next.js, TypeScript, JavaScript
- SPA-style dashboards and workflow-heavy admin apps
- HTML5, CSS3, responsive layouts, component-driven UI
- Forms, tables, filters, settings pages, status screens
- Customer portals, dealer portals, support/admin interfaces
- UX for operational users who need clarity and speed

Backend

- Node.js, Express, Next.js API routes
- REST APIs, webhooks, service-layer business logic
- JWT auth, role permissions, sessions, admin access control
- File uploads, document intake, notification workflows
- Background-style automation and integration workflows
- API design for frontend/backend integration

Databases / Data Modeling

- Postgres, SQLite, Supabase
- MongoDB / Mongoose
- Payload CMS data models

- Drizzle / Prisma-style relational modeling
- Workflow status models, audit trails, history records
- Customer, order, dealer, warranty, claim, support, product, shipping, and device records

DevOps / Delivery

- Git / GitHub
- Docker / Docker Compose familiarity
- Deployment notes and runbooks
- Vercel-style deployments
- CI/CD awareness and pipeline-friendly code organization
- Environment/config management and production debugging

Testing / Quality

- Practical unit/integration testing awareness
- Jest-style frontend/backend testing familiarity
- Manual verification plans for critical workflows
- Debugging production issues by isolating UI, API, data, auth, integration, and state layers
- Strong bias toward readable, maintainable code over clever abstractions

Additional Technical Range

- Python automation and scripting
- C#.NET tooling and interfaces
- C/C++ and embedded-adjacent work
- Flutter/Dart mobile apps
- BLE, CAN bus, LIN, RS-232/RS-485, firmware-update workflows
- Hardware-connected diagnostics, telemetry, and calibration workflows

PROFESSIONAL EXPERIENCE

Stealth Machine Tools / Stealth Batteries

Full-Stack Product Engineer, Manufacturing Software, Commerce/Admin, CRM, AI Support, Operations Tools Long-running company/product work

- Built production full-stack applications across Stealth Batteries, Stealth Machine Tools, AI support systems, CRM/order tools, customer portals, support workflows, warranty/claims workflows, and admin dashboards.
- Built Stealth Batteries commerce/admin platform using Next.js, React, TypeScript, Payload CMS, Postgres, Stripe, webhooks, Resend/Nodemailer, Vercel Blob, UPS/shipping APIs, dealer portals, affiliate workflows, sales-rep workflows, order management, warranty/support workflows, and internal dashboards.
- Built full-stack operational systems connecting products, customers, dealers, affiliates, sales reps, orders, shipping, support tickets, warranties, claims, approvals, notifications, and admin reporting into usable workflows.
- Built Stealth Machine Tools software across machine/customer/admin operations, support tooling, CRM workflows, machine records, training/support content, AI-assisted support, and manufacturing-adjacent business workflows.
- Built CAM/CAD-style and machine-tool support workflows around fiber lasers, cutting tools, pan brakes, customer onboarding, machine support, training, service records, and internal operations.
- Built GEO Command Center, a full-stack AI/search visibility platform with Next.js dashboards, app/API routes, FastAPI backend services, Supabase/Postgres data modeling, Drizzle schemas, worker-style processing, provider integrations, prompt/search audit workflows, citation tracking, report generation, analytics, and tests.
- Built AI support tooling with customer chat, onboarding, claim/warranty intake, internal workflow assistance, knowledge-base context, logging, analytics, ticketing, and human handoff paths.
- Built CRM/order/customer tracking systems with Node.js, Express, SQLite, JWT auth, role permissions, admin approval workflows, customer portals, OCR/document scanning, real-time updates, Twilio/Nodemailer notifications, audit trails, and deployment documentation.
- Built automation and dashboards that connected customer-facing portals, admin work, order state, approvals, document uploads, notifications, reporting, and support history into maintainable business workflows.

- Used Claude, Codex, Cursor-style AI development workflows, LM Studio, local LLMs, custom code-assistant pipelines, prompt engineering, architecture planning, codebase reverse engineering, scripted automation, and tool orchestration daily for implementation, code review, documentation, debugging, refactoring, codebase analysis, and test/checklist generation.

Burger Motorsports

Automotive Tuning, JB4Pro, CANFlex, Firmware-Adjacent Integration, Vehicle Software, BLE, CAN Bus 2019 onward

- Helped build and launch software around the JB4 tuning device ecosystem, including mobile application features, device-facing workflows, firmware-related flows, customer tools, diagnostics, support systems, and platform-specific behavior across many vehicle platforms and firmware combinations.
- Built JB4Pro mobile/device software using Flutter/Dart with BLE scan/connect behavior, connection recovery, notification parsing, ASCII/byte-level command handling, live gauges, datalogging, settings screens, user-adjustment interfaces, and developer diagnostics.
- Implemented firmware update workflows for JB4Pro-style hardware, including firmware download/manual HEX loading, bootloader entry, erase/write commands, block flashing, checksum validation, bootloader exit, progress state, and user-facing safety warnings.
- Built and maintained command/protocol logic for JB4 hardware, including settings retrieval, settings save, logging start/pause, map selection, E85/flex fuel configuration, WMI/meth settings, boost/fuel tables, AFR/meth/fuel pressure telemetry, and developer packet inspection.
- Developed CANFlex-style hardware software for fuel and sensor control: BLE device communication, ethanol content, fuel temperature, fuel pressure, CANbus output configuration, analog output modes, pressure sensor modes, calibration controls, firmware/version display, live gauges, logs, and race-car tuning workflows.
- Built CRM-backed support tooling that replaced manual email-based log review with structured customer submissions, vehicle log intake, AI-assisted parsing, diagnostic/tuning guidance, support history, and escalation paths.
- Worked close to embedded systems, firmware, custom hardware, and vehicle communication protocols, translating raw device/vehicle data into reliable UI state and supportable diagnostic workflows.

Together, my Stealth Machine Tools / Stealth Batteries work and Burger Motorsports work represent roughly 9 years of sustained production software experience across full-stack web applications, databases, mobile apps, AI integrations, business operations systems, hardware-connected software, vehicle protocols, BLE, CAN bus, firmware-adjacent workflows, diagnostics, and customer-facing tools.

Independent Vehicle / Hardware Software Projects

TurboLamik AWD Controller, CANFlex, JB4Pro, Vehicle Telemetry and Integration Tools

- Built a TurboLamik AWD controller project with ESP-IDF-oriented firmware scaffolding for BMW E90 + TurboLamik integration, including passive CAN listening on a shared 500 kbps bus, selected E90 CAN ID decoding, TurboLamik TCU frame decoding, normalized drivetrain/chassis signals, derived metrics, health/watchdog state, raw CAN capture, and shadow-mode AWD request computation.
- Reverse engineered TurboLamik ADX protocol behavior into a client-device spec, including 921600 baud serial assumptions, LOG1/LOG2 polling sequence, fixed 100-byte packet maps, gearbox torque, engine torque, input/output RPM, slip/clutch data, linear pressure, lockup, analog inputs, CAN torque fields, AWD activation, PWM output, steering angle, and error/safe-mode data.
- Built BMW E90 PT-CAN-style signal maps for throttle, engine RPM, wheel speeds, coolant/oil temperature, steering angle, brake proxy, handbrake, reverse, and validation notes separating confirmed decodes from provisional in-car validation items.
- Documented MaxxECU default CAN output integration for 11-bit IDs 0x520-0x542 at 500 kbit/s, including RPM, throttle, MAP, lambda, ignition, fuel duty, vehicle speed, slip, traction flags, brake/clutch flags, accelerations, oil pressure/temp, boost target, transmission temp, differential temp, and firmware-version-gated protocol differences.
- Designed Flutter telemetry/logging app shell for vehicle-side diagnostics, including device status, live dashboard, raw frame monitor, decoded signal monitor, capture control, saved sessions/export, and vehicle profile display.

Bayer

Technical / Analytical Testing Software Background 2015-2018

- Worked in analytical/scientific environments after studying biochemistry and software engineering.
- Built and supported custom code and analytical workflows for regulated testing and scientific/technical operations.
- Gained experience with careful documentation, data integrity, regulated processes, quality expectations, and technical troubleshooting.

SELECTED PROJECT HIGHLIGHTS

Stealth Batteries Commerce/Admin Platform

Built a production business platform connecting ecommerce, products, dealers, affiliates, sales reps, orders, shipping, support, warranty, dashboards, and Payload/Postgres admin tooling. The system tied customer-facing workflows to internal operations so the business could move faster without relying on manual email/status tracking.

Stealth Machine Tools / Manufacturing Software

Built software across the Stealth Machine Tools ecosystem, including CAM/CAD-style workflows, fiber laser and cutting-tool support, pan brake workflows, machine/customer/admin records, CRM/support tooling, training/support content management, customer onboarding, AI-assisted support, and full-stack web/admin systems. This work combined manufacturing context, operational software, customer support, and production business tooling.

GEO Command Center

Built a production-grade GEO / AI-search visibility platform for tracking how brands, products, pages, and topics appear across AI/search-style systems. This was a full product, not a demo: Next.js frontend, app/API routes, FastAPI backend services, Supabase/Postgres databases, Drizzle schema work, worker/background processing, provider integrations, prompt and content audits, citation/source tracking, recurring scans, report generation, analytics workflows, tests, and user-facing dashboards for decision-making.

Strong contribution areas:

- Owned the full-stack product shape across frontend dashboards, backend services, database models, worker pipelines, scripts, tests, and external AI/search connectors.
- Modeled tenants, brands, prompts, audits, citations, reports, visibility metrics, historical analytics, and review workflows.
- Designed recurring scan workflows with async processing, provider execution, result normalization, report generation, citation tracking, and dashboard review.
- Integrated AI/search providers into controlled, persisted product workflows with traceable outputs instead of one-off chatbot interactions.
- Built around real customer questions: where are we visible, what sources are cited, what content needs improvement, what changed over time, and what actions should the team take next.
- Published a sanitized full-stack source snapshot in the portfolio as geo-command-center-platform to show the frontend, backend, worker, database, test, and orchestration structure.

AI Support and Workflow Automation

Built AI-assisted customer and internal support systems with chat, onboarding, knowledge-base context, warranty/claims handling, ticketing, analytics, logging, escalation, and human handoff. These were practical workflow tools designed to reduce manual support load and help employees resolve customer issues faster.

Customer Tracking CRM

Built CRM/order workflow software with admin approvals, customer portal views, uploaded documents, OCR-style intake, audit trails, real-time updates, notifications, and role-aware workflows for messy business operations.

JB4Pro Vehicle Tuning Platform

Built mobile and hardware-connected software for JB4Pro tuning hardware, including BLE device discovery/connection, command handling, live telemetry, logging, firmware update flows, settings save/retrieval, map/user adjustment screens, E85/flex fuel workflows, WMI/meth configuration, boost/fuel tables, AFR/fuel pressure telemetry, and developer diagnostics.

CANFlex Fuel and Sensor Controller

Built CANFlex software for a fuel injector / E85 sensor / fuel pressure control ecosystem with BLE communication, ethanol/fuel-temperature/fuel-pressure gauges, CANbus and analog output configuration, pressure sensor modes, calibration, logs, firmware/version handling, and ECU/JB4 integration paths for race-car tuning.

TECHNOLOGIES

Frontend / SPA / UX: React, Next.js, TypeScript, JavaScript, semantic HTML5, CSS3, responsive UI, dashboards, customer portals, admin panels, forms, tables, workflow screens, status screens.

Backend / APIs: Node.js, Express, Next.js API routes, REST APIs, WebSockets, JWT auth, role permissions, service-layer logic, webhooks, file uploads, notifications, integration clients.

Databases / Data: Postgres, SQLite, MongoDB/Mongoose, Supabase, Payload CMS, Drizzle, Prisma-style modeling, workflow state, audit trails, reporting, operational dashboards.

Testing / Quality: Jest-style testing awareness, integration/manual verification, debugging, code review, logging, validation, edge-case handling, maintainability, technical debt cleanup.

DevOps / Delivery: Git, GitHub, Docker Compose, CI/CD awareness, deployment notes, Vercel-style hosting, environment configuration, production troubleshooting.

AI / Tools: Claude, Codex, Cursor-style workflows, LM Studio, local LLMs, Ollama-style local model workflows, Anthropic/OpenAI integrations, prompt engineering, AI-assisted code architecture, codebase reverse engineering, context-window strategy, prompt/context systems, guardrails, review loops, custom AI pipelines, coding agents, scripted codebase indexing, tool orchestration, AI support agents, document ingestion, OCR/document processing, analytics, automation, and human handoff.

Integrations: Stripe, UPS/shipping APIs, Resend/Nodemailer, Twilio, webhooks, email/SMS, payment/shipping/status workflows.

Hardware / Embedded-Adjacent: C, C#/ .NET, C++, ESP-IDF-oriented scaffolding, CAN bus, LIN bus awareness, BLE, RS-232/RS-485 familiarity, ECU/tuning workflows, raw CAN capture, diagnostics, datalogging, firmware update flows, bootloader workflows, HEX parsing, checksums.

Automation / Scripting: Python, PowerShell, FastAPI, internal scripts, support tooling, workflow automation, technical documentation.

EDUCATION

Biochemistry and Software Engineering coursework, 2012-2015. Studied biochemistry with a software engineering focus while preparing to build analytical testing and technical workflow software.

HOW I WORK

I am comfortable with "here is the problem, go figure it out" work. I ask scope questions early around users, data, edge cases, ownership, integrations, and production risk. I prefer simple architecture, clear data flow, useful validation, and code that another engineer can safely change. I communicate before going dark, use AI tools aggressively but responsibly, and leave notes/change logs so the system is easier to understand later.